

Daily Learning Notes

Python • SQL • Scikit-learn — Interview Prep Summary

1 Python — Department Summary

Task

Given employee dictionaries, generate a department-wise summary containing employee count, total salary, and average salary.

Important Pattern — Step 1: Group the Data

```
department_salaries = grp_by_dept(employees)

for dept, salaries in department_salaries.items():
    ...
```

Step 2: Build the Summary Dictionary per Department

```
depart = {}

depart["count"] = count
depart["total_salary"] = total_sal
depart["average_salary"] = avg_sal
```

Step 3: Store It

```
department_summary[dept] = depart
```

KEY THING LEARNED

Easy to confuse **dept** with **depart**:

- **dept** → department name / dictionary key
- **depart** → dictionary holding that department's summary

So `department_summary[dept] = depart` means:

- "IT" → {count, total_salary, average_salary}

INTERVIEW POINT

`append()` is for adding an item to a **list**:

```
my_list.append(item)
```

But `department_summary[dept] = depart` is used to create/store a **dictionary key-value pair**.

2 SQL — Customer Orders

Task

For each customer, calculate the number of orders, total amount, and average order amount. Then filter to customers who have at least 2 orders and spent more than 1000, sorted by total amount descending.

Final Query

```
SELECT customer_id,  
COUNT(order_id) AS total_orders,  
SUM(amount) AS total_amount,  
AVG(amount) AS avg_amount  
FROM Orders_1  
GROUP BY customer_id  
HAVING COUNT(order_id) >= 2  
AND SUM(amount) > 1000  
ORDER BY total_amount DESC;
```

IMPORTANT SQL PATTERN &MDASH; LOGICAL EXECUTION ORDER

FROM → GROUP BY → Aggregate functions → HAVING → ORDER BY

REMEMBER

WHERE → filters individual rows (before grouping)

HAVING → filters groups after GROUP BY

Since we filtered on COUNT(order_id) and SUM(amount) (aggregates), **HAVING** was the correct choice.

3 Scikit-learn — Logistic Regression

Task

Build a classification model to predict whether an employee gets promoted, based on **Age** and **Salary**, with target **Promotion**.

Complete Workflow

```
X, y
↓
train_test_split()
↓
LogisticRegression()
↓
fit()
↓
predict()
↓
Evaluation metrics
↓
New prediction
```

Split the Data

```
X_train, X_test, y_train, y_test = train_test_split(
X, y,
test_size=0.2,
random_state=42
)
```

Train

```
model = LogisticRegression()

model.fit(X_train, y_train)
```

Predict

```
y_pred = model.predict(X_test)
```

Evaluate

```
accuracy_score(y_test, y_pred)
precision_score(y_test, y_pred)
recall_score(y_test, y_pred)
f1_score(y_test, y_pred)
confusion_matrix(y_test, y_pred)
```

New Employee Prediction

```
new_employee = np.array([[27, 2700]])

prediction = model.predict(new_employee)
```

4 Interview Points From Today

fit() vs predict()

fit() — learns patterns/parameters from the training data.

predict() — uses the trained model to generate predictions on new data.

Why Train/Test Split?

We keep some data unseen during training so we can evaluate how the model performs on data it hasn't already learned from — this estimates real-world generalization.

Accuracy Isn't Always Enough

Metric	Value
Accuracy	95%
Precision	70%
Recall	40%

A model can have high accuracy but still perform poorly for the class we actually care about. For problems like **churn detection**, low recall means we're missing many actual churners — even if the headline accuracy looks great.